

```
#!pip install -q diffusers transformers accelerate safetensors
```

```
import torch
from diffusers import StableDiffusionPipeline
import matplotlib.pyplot as plt
from PIL import Image
import numpy as np
```

Flax classes are deprecated and will be removed in Diffusers v1.0.0. We recommend migrating to PyTorch classes or pinning your v
Flax classes are deprecated and will be removed in Diffusers v1.0.0. We recommend migrating to PyTorch classes or pinning your v

```
print("CUDA disponible:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
```

```
CUDA disponible: True
GPU: Tesla T4
```

```
model_id = "runwayml/stable-diffusion-v1-5"
device = "cuda"

prompt = "A highly detailed cinematic and futuristic fruit glowing in a cyberpunk laboratory, neon lights, 4k resolution"

num_steps = 20
seed = 42

capture_steps = [4, 10, 16, 20]
saved_latents = {}
```

modelo en GPU

```
pipe = StableDiffusionPipeline.from_pretrained(
    model_id,
    torch_dtype=torch.float16
).to(device)
```

```
pipe.safety_checker = None
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
```

```
warnings.warn(
model_index.json: 100% 541/541 [00:00<00:00, 20.7kB/s]
```

```
Download complete: 5.48G/? [01:15<00:00, 75.7MB/s]
```

```
Fetching 15 files: 100% 15/15 [01:05<00:00, 5.25s/it]
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
Loading pipeline components...: 100% 7/7 [00:24<00:00, 3.17s/it]
```

```
Loading weights: 100% 396/396 [00:04<00:00, 66.85it/s, Materializing param=visual_projection.weight]
```

```
StableDiffusionSafetyChecker LOAD REPORT from: /root/.cache/huggingface/hub/models--runwayml--stable-diffusion-v1-5/snapshots/45
Key | Status |
-----+-----+
vision_model.vision_model.embeddings.position_ids | UNEXPECTED |
```

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.

```
Loading weights: 100% 196/196 [00:02<00:00, 120.34it/s, Materializing param=text_model.final_layer_norm.weight]
```

```
CLIPTextModel LOAD REPORT from: /root/.cache/huggingface/hub/models--runwayml--stable-diffusion-v1-5/snapshots/451f4fe16113bff5a
Key | Status |
-----+-----+
text_model.embeddings.position_ids | UNEXPECTED |
```

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.

Fijar semilla

```
generator = torch.manual_seed(seed)
```

▼ Callback para capturar latentes

Guardamos los tensores en pasos 4, 10, 16 y 20.

Ejecucion de de difusión

```
def callback_fn(pipe, step, timestep, callback_kwargs):
    step_number = step + 1

    if step_number in capture_steps:
        latents = callback_kwargs["latents"]
        saved_latents[step_number] = latents.detach().clone()

    return callback_kwargs
_ = pipe(
    prompt=prompt,
    num_inference_steps=num_steps,
    generator=generator,
    callback_on_step_end=callback_fn,
)
```

100%

20/20 [00:03<00:00, 6.97it/s]

▼ Decodificar latentes con VAE

aplicar scaling_factor antes de decodificar

```
images = []

for step in capture_steps:
    latent = saved_latents[step]

    latent = latent / pipe.vae.config.scaling_factor

    with torch.no_grad():
        image = pipe.vae.decode(latent).sample

    image = (image / 2 + 0.5).clamp(0, 1)
    image = image.cpu().permute(0, 2, 3, 1).numpy()

    image = (image[0] * 255).astype(np.uint8)
    image = Image.fromarray(image)

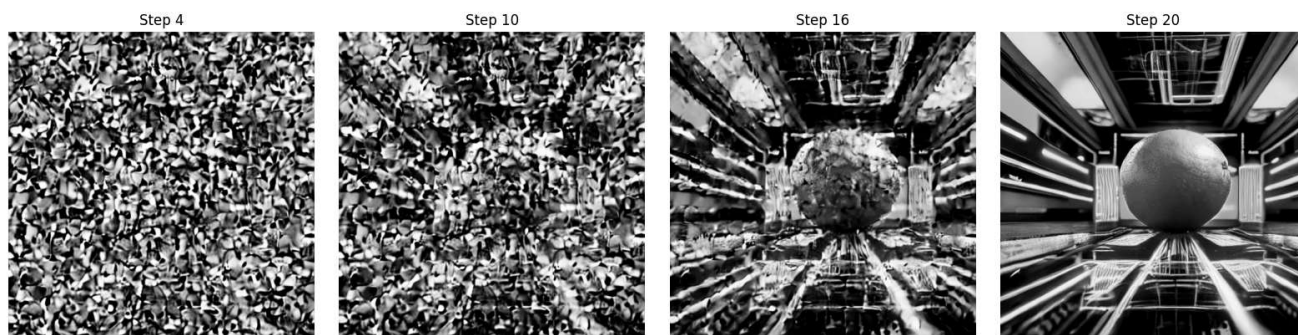
    images.append((step, image))
```

▼ Visualizar

```
fig, axes = plt.subplots(1, 4, figsize=(16, 4))

for ax, (step, img) in zip(axes, images):
    ax.imshow(img)
    ax.set_title(f"Step {step}")
    ax.axis("off")

plt.tight_layout()
plt.show()
```



En las etapas iniciales (alto ruido), la U-Net básicamente solo logra reconstruir bajas frecuencias, o sea la estructura gruesa de la imagen: la forma global del objeto, la silueta y una distribución básica de colores. En este punto el modelo ya “sabe” qué hay en la escena (el fruit, el laboratorio), pero todo se ve borroso porque el ruido todavía domina y elimina los detalles finos.

A medida que el ruido baja (etapas finales), la U-Net pasa a reconstruir altas frecuencias, refinando la imagen con bordes definidos, texturas, reflejos y detalles específicos como el glow del neón. Aquí ya no está decidiendo la estructura, sino puliendo la imagen para que se vea realista y con alta calidad.

En resumen, el proceso es claramente coarse-to-fine: primero se fija la geometría y composición global, y luego se añaden los detalles. El cross-attention guía todo el proceso, pero al inicio define la semántica general del prompt, mientras que al final controla los atributos finos como iluminación y estilo.

La clave es que la imagen no aparece de golpe, sino que se esculpe progresivamente en el espacio latente, separando naturalmente estructura (low freq) y detalle (high freq).